

EE-2003

Computer Organization and Assembly Language

Week 15 – Lecture 30

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 What Are Interrupts?
- 3 Software Interrupts
- 4 Interrupt Vector Table (IVT)
- 5 Reserved Interrupts
- 6 INT vs IRET
- 7 Summary
- 8 Hooking Interrupt 0 (Divide-by-Zero)

Quick Review: What was in the Last lecture?

- REPNE, REPE, SCASB and SCASW, LES instructions
- Finding Length of the strings

What Are Interrupts?

- Interrupts are events that occur outside the processor.
- They are asynchronous — independent of the currently executing instruction.
- They are unpredictable — you cannot know when they occur.
- Interrupts temporarily stop normal program flow.

Simple Idea: Interrupts let external events “grab” CPU attention immediately.

Types of Interrupts

- **Hardware Interrupts** Generated by external devices (keyboard, timer, hardware signals).
- **Software Interrupts** Generated using the INT instruction.

Hardware interrupts = real world events. Software interrupts = controlled jumps inside software.

Software Interrupts — Basic Concept

- $INT\ n$ behaves like a special FAR CALL.
- It pushes:
 - FLAGS
 - CS
 - IP
- Then jumps to the Interrupt Service Routine (ISR).
- ISR must end with IRET, not RET.

This ensures FLAGS are restored correctly.

What Happens During INT n

The CPU internally performs:

- 1 Push FLAGS
- 2 Push CS
- 3 Push IP
- 4 Clear IF (disable maskable interrupts)
- 5 Clear TF (disable single-step)
- 6 Load new CS:IP from IVT

This is an extended and safer form of a FAR CALL.

Interrupt Vector Table (IVT)

- Located at physical address **0000:0000**.
- Contains **256 entries**.
- Each entry = 4 bytes:
 - 2 bytes — offset
 - 2 bytes — segment
- Entry for interrupt n starts at:

$$n \times 4$$

Changing a vector = hooking an interrupt.

Why IVT Matters

- CPU does not know ISR address from INT instruction.
- Instead it “looks up” the address from IVT.
- Makes interrupt handling flexible:
 - Replace ISRs at runtime
 - Redirect interrupts easily

Intel Reserved Interrupts (0–4)

- **INT 0** – Divide by Zero (auto-generated)
- **INT 1** – Single Step (debug)
- **INT 2** – NMI (Non-maskable)
- **INT 3** – Breakpoint (1-byte opcode)
- **INT 4** – Overflow (via INTO)

These are reserved and generally not used by normal programs.

INT vs IRET

INT does:

- Push FLAGS
- Push CS
- Push IP
- Loads new CS:IP from IVT

IRET does:

- Pop IP
- Pop CS
- Pop FLAGS

Important: RET cannot restore FLAGS → ISR must end with IRET.

Summary of Concepts

- Interrupts allow external events to pause the CPU safely.
- Software interrupts use INT instructions.
- IVT at address 0 maps interrupt numbers to handlers.
- CPU automatically saves FLAGS, CS, IP during INT.
- ISRs must return using IRET.
- Intel reserved interrupts (0–4) have special meanings.

Interrupt Vector Table (IVT)

- Located at physical address **0000:0000**.
- Contains 256 entries (0–255), each 4 bytes.
- For interrupt n :

Offset at $n \times 4$, Segment at $n \times 4 + 2$

INT No.	Offset Addr	Segment Addr
0	0000	0002
1	0004	0006
2	0008	000A
⋮	⋮	⋮

Before vs After Hooking INT 0

Before Hooking:

- INT 0 jumps to BIOS default divide-error handler.
- System shows standard “Divide Error”.

After Hooking:

- We overwrite IVT entry of INT 0.
- INT 0 now jumps to our custom ISR `myisrfor0`.
- Our own screen message is displayed.

Flow of INT → ISR → IRET

- Program runs normally.
- DIV instruction causes overflow → CPU triggers INT 0.
- CPU pushes FLAGS, CS, and IP.
- CPU loads new CS:IP from IVT.
- Executes custom ISR.
- IRET restores original FLAGS, CS, IP.

Program → INT 0 → Push Registers → Jump to ISR → IRET → Return

Custom ISR for INT 0 — Part 1 (Example 8.1)

```
myisrfor0:
    push ax
    push bx
    push cx
    push dx
    push si
    push di
    push bp
    push ds
    push es
    push cs
    pop  ds      ; DS = CS (access data)
    call clrscr
```

Custom ISR for INT 0 — Part 2

```
mov ax, 30
push ax      ; x-position
mov ax, 20
push ax      ; y-position
mov ax, 0x71
push ax      ; color attr
mov ax, message
push ax      ; string addr
call printstr

pop es
pop ds
pop bp
pop di
pop si
pop dx
pop cx
pop bx
pop ax
iret
```

Installing Our ISR Into IVT

```
start:
    xor ax, ax
    mov es, ax          ; ES = 0000 (IVT)

    mov word [es:0*4], myisrfor0 ; new offset
    mov [es:0*4+2], cs         ; new segment

    call genint0              ; generate divide error

    mov ax, 0x4c00
    int 0x21
```

Generating Divide Error (INT 0)

```
genint0:
    mov ax, 0x8432
    mov bl, 2
    div bl          ; result too big → INT 0
    ret
```

Why hang?

- INT 0 returns to the SAME DIV instruction.
- DIV fails again → INT 0 again.
- Infinite loop.

Thank You!

Any Questions?

- Today we learned how the **8088 interrupt mechanism** works and how software interrupts behave like extended far calls.
- We explored the **Interrupt Vector Table (IVT)** and how each interrupt maps to its handler.
- We practiced **hooking INT 0** by modifying its vector to point to our own custom divide-by-zero ISR.
- The ISR demonstrated saving registers, clearing the screen, printing a message, and returning with IRET.
- We also saw how a **DIV overflow automatically triggers INT 0**, causing repeated interrupts when returned to the same instruction.

"Interrupts give us a powerful, flexible way to redirect program flow and handle events at the hardware level."